[Topic 1] Shell Programming

[Subtopics] Argument passing, Taking Inputs, Conditionals

Lab Manual:
https://drive.google.com/file/d/1OooOzJVnVMrVWloPWnbrBItBnOpAONSN/view?usp=sharing

---

Sample Questions:

#1 Create a file calc.sh. It will take 3 arguments. 1st and 2nd arguments are operands and the third argument is any of the operation (+, -, *, /) to be performed. Show the respected outputs and if none of the operations, show that the operation is not supported.

```
#! /bin/bash

if [ $3 = + ]; then
	echo $(($1+$2))
elif [ $3 = - ]; then
	echo $(($1-$2))
elif [ $3 = * ]; then
	echo $(($1*$2))
elif [ $3 = / ]; then
	echo $(($1/$2))
else
	echo Operation not supported
fi
```

[Topic 2] Linux Process Management

[Subtopics] fork(),

Lab Manual:
https://drive.google.com/file/d/15jB7Llda4kMIL_9t91yxNs1YerHZXm6r/view?usp=sharing

---

Sample Questions:

#1 Write a C Program where the parent creates a child using fork() then continues printing odd numbers and the child prints odd numbers. The numbers are in range 1-10.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
	pid_t pid = fork();

	if (pid < 0) {
	// Error handling
		perror("fork failed");
		return 1;
	} else if (pid == 0) {
		// Child process
		for (int i = 1; i <= 10; i++) {
			if (i % 2 == 0) {
				printf("Child: %d\n", i);
				usleep(100000); // Sleep 100ms
			}
		}
	} else {
	// Parent process
		for (int i = 1; i <= 10; i++) {
			if (i % 2 != 0) {
				printf("Parent: %d\n", i);
				usleep(100000); // Sleep 100ms
			}
		}
	}

	return 0;
}
```

[Topic 3] Xinu Process Management

[Subtopics] Process Creation, Semaphore

Lab Manual:
https://drive.google.com/file/d/1kw-BiEtT-7ryd6mbtnq1MCYyPk8ZM4Gh/view?usp=sharing
Book Chapter:
Chapter 2

Sample Questions:
All code from Chapter 2

[Topic 4] Xinu IPC

[Subtopics] Port

Lab Manual:
https://drive.google.com/file/d/1kw-BiEtT-7ryd6mbtnq1MCYyPk8ZM4Gh/view?usp=sharing
Book Chapter:
Chapter 11

Sample Questions:
#1 Write a program in Xinu, where the main process creates a port and 2 processes- producer and consumer. Producer puts a value in range 1-10 one by one in port and Consumer reads them one by one and prints them.

```
#include <xinu.h>

int prod2(int32 ptid);
int cons2(int32 ptid);

shellcmd xsh_hello(int nargs, char *args[]) {
	ptinit(100); // Important! You need to manually call this here!
	int32 ptid = ptcreate(10);

	if (ptid == SYSERR) {
		kprintf("Failed to create port\n");
		return 1;
	}

	resume(create(cons2, 1024, 20, "consumer", 1, ptid));
	resume(create(prod2, 1024, 20, "producer", 1, ptid));

	return 0;
}

int prod2(int32 ptid) {
	int i;
	for(i=0; i<10; i++) {
		int32 msg = ptsend(ptid, i);
		resched_cntl(DEFER_START);
		if (msg == SYSERR) {
			kprintf("Producer: Failed to send message\n");
		} else {
			kprintf("Producer: Sent message %d\n", i);
		}
		resched_cntl(DEFER_STOP);
```

```
	}
	return 0;
}

int cons2(int32 ptid) {
	int i;
	for(i=0; i<10; i++) {
		int32 msg = ptrecv(ptid);
		resched_cntl(DEFER_START);
		if (msg == SYSERR) {
			kprintf("Consumer: Failed to receive message\n");
		} else {
			kprintf("Consumer: Received message %d\n", msg);
		}
		resched_cntl(DEFER_STOP);
	}
	return 0;
}
```